Министерство образования Республики Беларусь
Учреждение образования
«Брестский Государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине «Конструирование программного обеспечения»
Тема: «Создание системы сохранений и загрузок»

Выполнил:
Студент 2 курса
250966
Группы ПО-17
Фещук А.А.
Проверила:
Кулик А.Д.

Брест 2026

**Цель работы:** написать логику для сохранения данных программы и последующей их загрузки.

1. Реализовать логику сохранения в 3 разных слота. Необходимо также отображать время сохранения каждой из игр. Все слоты сохранения реализовать в одном файле.

## Код программы:

```
#define _CRT_SECURE_NO_WARNINGS // Отключает ошибку C4996 с localtime в Visual Studio
#include <iostream>
#include <fstream>
#include <queue>
#include <vector>
#include <string>
#include <limits>
#include <algorithm>
#include <ctime>
#include <sstream>
#include <iomanip>

using namespace std;

// ==================== КОНФИГУРАЦИЯ ====================
struct GameConfig {
    int citySize = 10, budget = 1000, population = 100;
};

class ConfigManager {
    GameConfig cfg;
    queue<int> sizeOptions, budgetOptions, popOptions;

    void loadData(const string& filename, queue<int>& q, const vector<int>& defaults)
{
        ifstream in(filename);
        int v;
        if (in.good()) { while (in >> v) q.push(v); return; }
        ofstream out(filename);
        for (int x : defaults) { out << x << "\n"; q.push(x); }
    }

    int getNextValue(queue<int>& q, int current) {
        if (q.empty()) return current;
        int firstVal = q.front(), nextVal = firstVal;
        for (size_t i = 0; i < q.size(); ++i) {
            int v = q.front(); q.pop(); q.push(v);
            if (v == current) { q.pop(); nextVal = q.empty() ? firstVal : q.front();
q.push(nextVal); break; }
        }
        return nextVal;
    }

public:
    ConfigManager() {
        loadData("city_size_values.txt", sizeOptions, { 10, 20, 30 });
        loadData("budget_values.txt", budgetOptions, { 1000, 5000, 10000 });
        loadData("population_values.txt", popOptions, { 100, 500, 1000 });
        ifstream in("settings.txt");
        if (!(in >> cfg.citySize >> cfg.budget >> cfg.population)) saveConfig();
    }
    GameConfig& getSettings() { return cfg; }
    void saveConfig() {
        ofstream out("settings.txt");
        out << cfg.citySize << "\n" << cfg.budget << "\n" << cfg.population << "\n";
    }
    void changeSize() { cfg.citySize = getNextValue(sizeOptions, cfg.citySize);
saveConfig(); }
    void changeBudget() { cfg.budget = getNextValue(budgetOptions, cfg.budget);
saveConfig(); }
    void changePop() { cfg.population = getNextValue(popOptions, cfg.population);
saveConfig(); }
};
```

```
// ==================== ИГРОВЫЕ СУЩНОСТИ ====================
class CityObject {
public:
    string title;
    int price, weeklyIncome, citizenBonus, moodBonus;
    CityObject(string t, int p, int inc, int pop, int mood)
        : title(t), price(p), weeklyIncome(inc), citizenBonus(pop), moodBonus(mood) {}

    void display(int index) const {
        cout << " [" << index << "] " << title
            << " | Стоимость: " << price
            << " | Прибыль: " << weeklyIncome
            << " | +Жители: " << citizenBonus
            << " | Настроение: " << (moodBonus >= 0 ? "+" : "") << moodBonus << "\n";
    }
};

class CitySimulation {
public:
    int maxBuildings, currentBudget, totalPopulation, happinessLevel = 50;
    vector<CityObject> constructedObjects;

    CitySimulation(int limit, int cash, int people)
        : maxBuildings(limit), currentBudget(cash), totalPopulation(people) {}

    bool construct(const CityObject& obj) {
        if (currentBudget < obj.price) {
            cout << "Ошибка: не хватает денег на " << obj.title << "!\n";
            return false;
        }
        if ((int)constructedObjects.size() >= maxBuildings) {
            cout << "Ошибка: в городе больше нет места (лимит: " << maxBuildings <<
").\n";
            return false;
        }
        currentBudget -= obj.price;
        totalPopulation += obj.citizenBonus;
        happinessLevel = min(100, max(0, happinessLevel + obj.moodBonus));
        constructedObjects.push_back(obj);
        cout << "Успешно возведено: " << obj.title << "\n";
        return true;
    }

    void printStatus() const {
        cout << "--- ОТЧЕТ О ГОРОДЕ ---\n";
        cout << "Земля: " << maxBuildings << " мест (занято: " <<
(int)constructedObjects.size() << ")\n";
        cout << "Казна: " << currentBudget << "\n";
        cout << "Граждане: " << totalPopulation << "\n";
        cout << "Уровень счастья: " << happinessLevel << "/100\n";
        if (!constructedObjects.empty()) {
            cout << "Инфраструктура:\n";
            for (size_t i = 0; i < constructedObjects.size(); ++i)
                cout << "  - " << constructedObjects[i].title << "\n";
        }
    }

    int simulateWeek() {
        int profit = 0;
        for (auto& b : constructedObjects) profit += b.weeklyIncome;
        int taxes = totalPopulation / 10;
        double multiplier = 0.5 + happinessLevel / 100.0;
        int finalIncome = (int)((profit + taxes) * multiplier);
        currentBudget += finalIncome;
        if (happinessLevel >= 60) totalPopulation += totalPopulation / 20;
        else if (happinessLevel <= 30) totalPopulation -= totalPopulation / 25;
        return finalIncome;
    }
};

class PlayerActions {
public:
    int movesLeft = 5, currentWeek = 0;
    int builtCount = 0;
    int festivalsCount = 0;
```

```
    void resetMoves() { movesLeft = 5; }
    bool canMove() const { return movesLeft > 0; }
    void spendMove() { --movesLeft; }
};

// ==================== СИСТЕМА СОХРАНЕНИЙ (ВАРИАНТ 1) ====================
class SaveSlotManager {
    const string SAVE_FILE = "saves.txt";
    const char DELIM = '#';

public:
    string getCurrentTimeStr() {
        time_t now = time(0);
        tm *ltm = localtime(&now);
        char buffer[80];
        strftime(buffer, sizeof(buffer), "%Y-%m-%d %H:%M:%S", ltm);
        return string(buffer);
    }

    void saveGame(int slot, const CitySimulation& game, const PlayerActions& player) {
        vector<string> lines(3, "");
        ifstream fin(SAVE_FILE);
        if (fin.is_open()) {
            for (int i = 0; i < 3; ++i) {
                if (!getline(fin, lines[i])) break;
            }
        }
        fin.close();

        stringstream ss;
        ss << (slot - 1) << DELIM
           << getCurrentTimeStr() << DELIM
           << player.currentWeek << DELIM
           << game.currentBudget << DELIM
           << game.totalPopulation << DELIM
           << game.happinessLevel << DELIM
           << player.builtCount << DELIM
           << player.festivalsCount << DELIM
           << player.movesLeft << DELIM;

        for (size_t i = 0; i < game.constructedObjects.size(); ++i) {
            ss << game.constructedObjects[i].title;
            if (i + 1 < game.constructedObjects.size()) ss << ",";
        }

        lines[slot - 1] = ss.str();

        ofstream fout(SAVE_FILE);
        for (int i = 0; i < 3; ++i) {
            fout << lines[i] << "\n";
        }
        cout << "Игра успешно сохранена в слот " << slot << "!\n";
    }

    bool loadGame(int slot, CitySimulation& game, PlayerActions& player, const
vector<CityObject>& catalog) {
        ifstream fin(SAVE_FILE);
        if (!fin.is_open()) return false;

        string line;
        while (getline(fin, line)) {
            if (line.empty()) continue;
            stringstream ss(line);
            string token;
            vector<string> tokens;
            while (getline(ss, token, DELIM)) {
                tokens.push_back(token);
            }
            if (tokens.size() >= 9 && stoi(tokens[0]) == (slot - 1)) {
                try {
                    player.currentWeek = stoi(tokens[2]);
                    game.currentBudget = stoi(tokens[3]);
                    game.totalPopulation = stoi(tokens[4]);
                    game.happinessLevel = stoi(tokens[5]);
                    player.builtCount = stoi(tokens[6]);
                    player.festivalsCount = stoi(tokens[7]);
                    player.movesLeft = stoi(tokens[8]);
```

```
                    game.constructedObjects.clear();
                    if (tokens.size() == 10 && !tokens[9].empty()) {
                        stringstream obj_ss(tokens[9]);
                        string obj_name;
                        while (getline(obj_ss, obj_name, ',')) {
                            for (const auto& c : catalog) {
                                if (c.title == obj_name) {
                                    game.constructedObjects.push_back(c);
                                    break;
                                }
                            }
                        }
                    }
                    return true;
                } catch (...) {
                    return false;
                }
            }
        }
        return false;
    }

    void printSavesInfo() {
        ifstream fin(SAVE_FILE);
        if (!fin.is_open()) {
            cout << "\nСохранений пока нет.\n";
            return;
        }
        string line;
        bool hasSaves = false;
        cout << "\n--- ДОСТУПНЫЕ СОХРАНЕНИЯ ---\n";
        for (int i = 0; i < 3; ++i) {
            if (getline(fin, line) && !line.empty()) {
                stringstream ss(line);
                string token;
                vector<string> tokens;
                while (getline(ss, token, DELIM)) tokens.push_back(token);
                if (tokens.size() >= 2) {
                    cout << "Слот " << (i + 1) << " | Время: " << tokens[1] << "\n";
                    hasSaves = true;
                }
            }
        }
        if (!hasSaves) cout << "Сохранений пока нет.\n";
        cout << "----------------------------\n";
    }
};

// ==================== ВСПОМОГАТЕЛЬНЫЕ ФУНКЦИИ ====================
void cls() {
#ifdef _WIN32
    system("cls");
#else
    system("clear");
#endif
}

void pause() {
    cout << "\n[Нажмите Enter для продолжения...]";
    cin.get();
}

int getUserInput(const string& prompt) {
    int val;
    while (true) {
        cout << prompt;
        if (cin >> val) { cin.ignore(10000, '\n'); return val; }
        cin.clear(); cin.ignore(10000, '\n');
        cout << "Неверный формат. Попробуйте снова.\n";
    }
}

vector<CityObject> getCatalog() {
    return {
        {"Жилой комплекс", 200, 20, 30, 5},
        {"Торговый центр", 300, 60, 5, 3},
```

```
        {"Завод", 500, 120, 10, -5},
        {"Сквер", 150, 5, 0, 15},
        {"Гимназия", 400, 30, 5, 10},
        {"Оперный театр", 350, 40, 0, 18}
    };
}

// ==================== ОСНОВНОЙ ЦИКЛ ИГРЫ ====================
void playGame(CitySimulation& game, PlayerActions& player, SaveSlotManager& saveMgr) {
    vector<CityObject> shop = getCatalog();
    const int TOTAL_WEEKS = 6;

    while (true) {
        if (player.movesLeft == 5) {
            player.currentWeek++;
            if (player.currentWeek > TOTAL_WEEKS) break;
            cout << "\n--------------------------------------------\n";
            cout << " ТЕКУЩАЯ НЕДЕЛЯ: " << player.currentWeek << " из " << TOTAL_WEEKS
<< "\n";
            cout << "--------------------------------------------\n";
        }

        while (player.canMove()) {
            cls();
            game.printStatus();
            cout << "\nХодов осталось: " << player.movesLeft << "\n";
            cout << "1. Строительство\n";
            cout << "2. Организовать праздник (+10 счастья, -100 монет)\n";
            cout << "3. Завершить неделю\n";
            cout << "4. Сохранить игру\n";
            int action = getUserInput("Ваш выбор: ");

            if (action == 1) {
                cls();
                cout << "=== ДОСТУПНЫЕ ПРОЕКТЫ ===\n";
                for (size_t i = 0; i < shop.size(); ++i) shop[i].display((int)i + 1);
                cout << " 0. Вернуться\n";
                int choice = getUserInput("Что будем строить? ");
                if (choice >= 1 && choice <= (int)shop.size()) {
                    if (game.construct(shop[choice - 1])) {
                        player.spendMove();
                        player.builtCount++;
                    }
                    pause();
                }
            }
            else if (action == 2) {
                if (game.currentBudget >= 100) {
                    game.currentBudget -= 100;
                    game.happinessLevel = min(100, game.happinessLevel + 10);
                    cout << "Праздник удался! Счастье выросло.\n";
                    player.spendMove();
                    player.festivalsCount++;
                }
                else cout << "В казне пусто, праздник отменяется.\n";
                pause();
            }
            else if (action == 3) {
                player.movesLeft = 0;
            }
            else if (action == 4) {
                int slot = getUserInput("Выберите слот для сохранения (1-3): ");
                if (slot >= 1 && slot <= 3) {
                    saveMgr.saveGame(slot, game, player);
                } else {
                    cout << "Неверный номер слота!\n";
                }
                pause();
            }
            else {
                cout << "Такого пункта нет!\n";
                pause();
            }
        }

        if (player.currentWeek <= TOTAL_WEEKS) {
            int weekProfit = game.simulateWeek();
```

```
            cout << "\n=== ИТОГИ НЕДЕЛИ " << player.currentWeek << " ===\n";
            cout << "Прибыль: " << weekProfit << "\n";
            cout << "Остаток в казне: " << game.currentBudget << "\n";
            cout << "Численность: " << game.totalPopulation << "\n";
            cout << "Счастье: " << game.happinessLevel << "/100\n";
            pause();
            player.movesLeft = 5;
        }
    }

    cls();
    cout << "=== СИМУЛЯЦИЯ ЗАВЕРШЕНА ===\n";
    game.printStatus();

    int finalScore = game.currentBudget + game.totalPopulation * 2 +
game.happinessLevel * 5;

    cout << "\n============================================\n";
    cout << " ВЕРДИКТ ГОРОДСКОГО СОВЕТА\n";
    cout << "============================================\n";

    // Убрали эмодзи, чтобы Visual Studio не выдавала ошибку кодировки
    if (player.builtCount == 0 && player.festivalsCount == 0) {
        cout << "[!] СНЯТ С ДОЛЖНОСТИ МЭРА!\n";
        cout << "Вы ничего не сделали для города.\n";
    } else if (player.builtCount <= 3) {
        cout << "[!] ВЫ ЕДВА СПРАВИЛИСЬ С ОБЯЗАННОСТЯМИ\n";
        cout << "Город развивался слишком медленно.\n";
    } else if (game.happinessLevel <= 20) {
        cout << "[!] ГОРОД В УПАДКЕ!\n";
        cout << "Вы забыли о счастье горожан.\n";
    } else if (player.builtCount >= 10 && game.happinessLevel >= 70) {
        cout << "[+] ВЫ - ВЕЛИКИЙ МЭР!\n";
        cout << "Город процветает благодаря вам!\n";
    } else {
        cout << "[+] СТАБИЛЬНЫЙ РУКОВОДИТЕЛЬ\n";
        cout << "Вы неплохо справились.\n";
    }
    cout << "============================================\n";
    cout << "\nВаш итоговый рейтинг: " << finalScore << "\n";
    pause();
}

void runSimulation(const GameConfig& settings, SaveSlotManager& saveMgr) {
    cls();
    cout << ">>> СТАРТ СИМУЛЯЦИИ: «ГОРОД МЕЧТЫ» <<<\n\n";
    cout << "Добро пожаловать на пост мэра!\n";
    cout << "Ваши стартовые условия:\n";
    cout << "  Площадь: " << settings.citySize << "\n";
    cout << "  Стартовый капитал: " << settings.budget << "\n";
    cout << "  Жители: " << settings.population << "\n";

    CitySimulation game(settings.citySize, settings.budget, settings.population);
    PlayerActions player;
    playGame(game, player, saveMgr);
}

void loadGameMenu(SaveSlotManager& saveMgr) {
    cls();
    cout << "=== ЗАГРУЗКА ИГРЫ ===\n";
    saveMgr.printSavesInfo();
    int slot = getUserInput("Выберите слот для загрузки (1-3) или 0 для отмены: ");
    if (slot >= 1 && slot <= 3) {
        GameConfig defaultCfg;
        CitySimulation game(defaultCfg.citySize, defaultCfg.budget,
defaultCfg.population);
        PlayerActions player;
        vector<CityObject> catalog = getCatalog();

        if (saveMgr.loadGame(slot, game, player, catalog)) {
            cout << "Игра успешно загружена из слота " << slot << "!\n";
            pause();
            playGame(game, player, saveMgr);
        } else {
            cout << "Слот пуст или поврежден.\n";
            pause();
        }
```

```
    }
}

#ifndef UNIT_TEST
int main() {
    setlocale(LC_ALL, "Russian");
    ConfigManager config;
    SaveSlotManager saveMgr;
    int menuChoice = 0;

    while (menuChoice != 5) {
        cls();
        cout << "====================================\n";
        cout << " ДОБРО ПОЖАЛОВАТЬ В ГОРОД МЕЧТЫ\n";
        cout << "====================================\n";
        cout << "1. Новая игра\n";
        cout << "2. Загрузить игру\n";
        cout << "3. Параметры\n";
        cout << "4. Информация\n";
        cout << "5. Выход\n";
        cout << "====================================\n";
        menuChoice = getUserInput("Выберите действие: ");

        if (menuChoice == 1) runSimulation(config.getSettings(), saveMgr);
        else if (menuChoice == 2) loadGameMenu(saveMgr);
        else if (menuChoice == 3) {
             cout << "Настройки пока недоступны в этой версии.\n";
             pause();
        }
        else if (menuChoice == 4) {
            cls();
            cout << "=== О ПРОЕКТЕ ===\n";
            cout << "Игра: «Город мечты»\n";
            cout << "Версия: 3.0 (ЛР4)\n";
            cout << "Автор: Фещук Артём\n";
            pause();
        }
        else if (menuChoice == 5) cout << "Завершение работы...\n";
        else { cout << "Ошибка ввода!\n"; pause(); }
    }
    return 0;
}
#endif
```

**0. Алгоритм работы написанной программы**
Программа «Город мечты» представляет собой консольную пошаговую игру-симулятор, алгоритм работы которой следующий:

Инициализация: При запуске программа загружает настройки из файлов (settings.txt и файлов со значениями параметров). Если файлов нет, они создаются с параметрами по умолчанию.

Главное меню: Запускается бесконечный цикл, который выводит на экран меню с вариантами действий (Новая игра, Загрузить, Параметры, Информация, Выход).

Новая игра: При выборе этого пункта создается объект симуляции с начальными параметрами. Запускается игровой цикл (длится 6 недель). В каждую неделю у игрока есть 5 ходов, которые он может потратить на строительство зданий или проведение праздников.

Сохранение (ЛР4): В процессе игры игрок может выбрать пункт «Сохранить игру». Программа собирает текущее состояние (номер недели, бюджет, население, счастье, количество оставшихся ходов и список построенных зданий), сериализует их в строку с разделителями и записывает в выбранный слот (1, 2 или 3) в файл saves.txt, прикрепляя текущую дату и время.

Загрузка (ЛР4): При выборе загрузки программа считывает файл saves.txt, находит нужный слот, парсит строку обратно в переменные, восстанавливает состояние объектов игры и продолжает игровой цикл ровно с того момента, где он был прерван.

Завершение: По истечении 6 недель программа подсчитывает итоговый рейтинг, выводит вердикт мэра и возвращается в главное меню.

**1. Тестирование. Виды и алгоритмы тестирования**
Тестирование — это процесс исследования программного обеспечения с целью проверки соответствия требований к нему и выявления дефектов.
Виды тестирования:
По уровню:
Модульное (Unit): проверка отдельных функций или классов (именно такой тип реализован в файле test.cpp).

Интеграционное: проверка взаимодействия модулей друг с другом.

Системное: проверка всей программы целиком на соответствие требованиям.

По доступу к коду:

«Черный ящик»: тестировщик не знает внутреннего устройства, проверяет только входные и выходные данные.

«Белый ящик»: тестирование с полным знанием исходного кода.

По автоматизации: Ручное и автоматизированное (как запуск скриптов через GitHub Actions).
Алгоритмы (подходы):

«Сверху вниз»: сначала тестируются главные модули, затем вспомогательные.

«Снизу вверх»: сначала тестируются базовые функции, затем система в целом.

---

**2. Защита программного продукта**
Защита ПО направлена на предотвращение несанкционированного использования, копирования или изменения программы.

Правовые методы: Лицензирование (проприетарные лицензии, Open Source — GPL, MIT), авторское право, патенты.

Технические методы:

Обфускация кода: приведение исходного кода к виду, который сложно прочитать человеку (запутывание имен переменных, вставка "мертвого" кода).

Шифрование: защита исполняемых файлов и данных.

Аппаратная защита: использование USB-ключей (dongles).

Привязка к оборудованию: программа работает только на конкретном ПК (по MAC-адресу или серийному номеру жесткого диска).

Серверная активация: проверка лицензии через интернет (DRM).

**3. Коллективная разработка ПО. Роли**

Коллективная разработка — это процесс создания ПО командой специалистов. Для успешной работы используются методологии (Agile, Scrum, Waterfall) и системы контроля версий (например, Git, который мы используем).
Основные роли в команде:

Product Manager (PM): Отвечает за то, что нужно сделать (требования, ценность для пользователя).
Project Manager: Отвечает за то, как и когда будет сделана работа (сроки, ресурсы, риски).

Team Lead / Architect: Технический лидер, принимает ключевые архитектурные решения.

Разработчики (Developers): Пишут код (Frontend, Backend, Fullstack).

QA-инженеры (Тестировщики): Проверяют качество продукта, ищут баги.

UI/UX Дизайнеры: Проектируют внешний вид и удобство использования интерфейса.

DevOps-инженер: Настраивает серверы, автоматизирует сборку и развертывание (CI/CD).

**4. Спецификация и планирование**

Спецификация (Техническое задание): Документ, который четко описывает, что должна делать программа. Включает в себя: цели, функциональные требования (что делать), нефункциональные требования (как быстро, безопасно и т.д.), описание интерфейса и ограничения.

Планирование: Процесс разбиения большого проекта на более мелкие задачи. Включает в себя:

Оценку сложности задач (например, с помощью Story Points).

Составление дорожной карты (Roadmap).

Управление рисками (что делать, если разработчик заболеет или сроки сорвутся).

Распределение ресурсов (кто какую задачу будет делать).

**5. Документирование и анализ ошибок**

Документирование: Создание документации, облегчающей понимание и поддержку кода.
Бывает:

Пользовательская (инструкции для игроков).

Техническая (описание API, архитектуры, комментарии в коде — как в наших функциях saveGame и loadGame).
Управленческая (отчеты о проделанной работе).
Анализ ошибок (Bug Tracking): Процесс выявления, регистрации и исправления дефектов.
Ошибки заносятся в специальные трекеры (Jira, GitHub Issues).
Им присваивается приоритет (критический баг, блокирующий игру, или мелкий визуальный недочет).
Проводится Root Cause Analysis (RCA) — поиск первопричины ошибки (например, почему в файле сохранения записался неверный ход), чтобы не допустить её появления в будущем.